`scripts/autonomous-qa/lib-emulator.sh` — User Guide

Last verified: 2026-07-26 (LVA-018 script-docs backfill cycle) **Inheritance:** HelixConstitution §11.4.18 (script docs); Lava §6.AG / §6.AH / §6.X (containerized emulators), §6.R (no-hardcoding, slirp exemption) **Classification:** project-specific

Overview

Sourceable library of thin Lava glue replicating `submodules/containers/pkg/emulator/containerized.go` to boot the §6.X **containerized KVM Android emulator** image and wire host adb. The emulator process runs INSIDE a podman container (image built from the submodule Containerfile), NEVER host-direct, NEVER a live device. The image bundles the AVD `default`, an adb keypair, and a socat bridge (container `0.0.0.0:5575` → `127.0.0.1:5555` adbd).

Functions

Function	Purpose
`emu_pick_port`	Pick a free TCP port on loopback
`emu_boot [avd]`	Boot the container (cold boot); prints the container name; persists state
`emu_authorize_adb <container>`	Copy the image's baked adb key out and point the host adb client at it (`ADB_VENDOR_KEYS`)
`emu_connect <adb_port>`	`adb connect`; prints the adb serial (`127.0.0.1:<port>`)
`emu_wait_boot <serial> [timeout_secs]`	Poll `sys.boot_completed` + `pm` ready (default 360 s); dumps container logs on timeout
`emu_fix_network <serial>`	Ensure the guest has an active default network (see below)
`emu_cleanup_orphans`	

Function	Purpose
	Remove leftover `lava-emu-*` containers from an interrupted run
`emu_teardown [container]`	Remove the container + the state file

State + configuration

- State file: `scripts/autonomous-qa/.emu-state` (`CONTAINER` / `ADB_PORT` / `CONSOLE_PORT`) — written by `emu_boot`, removed by `emu_teardown`.
- `EMU_IMAGE` (env, default `ghcr.io/vasic-digital/lava-android-emulator:api34-x86_64`) — the emulator image.
- Container run flags: `--userns=keep-id --device /dev/kvm`, host-loopback port publishing only, `ANDROID_COLD_BOOT=true`.

`emu_fix_network` — the eth0/ndc default-network fix

On some hosts (observed: podman 4.9.3 on thinker.local) the emulator boots with eth0 DOWN and Android never registers it as a framework network — apps see “Network unreachable” because Android routes per-network via netd mark-based tables. The function (root via `su 0` inside the guest) brings eth0 up and adds it to the default physical network (netId 100) through `ndc`, retrying up to 8 times until the slirp gateway pings. Self-gating + idempotent: if the gateway already answers it no-ops; it never fails the run — the iteration records the REAL outcome (anti-bluff: a stalled onboarding shows as SKIP/FAIL).

§6.R note: the `10.0.2.x` literals are platform-FIXED Android-emulator QEMU user-mode (slirp) constants hardwired by the emulator — `10.0.2.2` = host-loopback alias, `10.0.2.3` = built-in DNS, `10.0.2.15` = the guest NIC. They are the emulator equivalent of `127.0.0.1` and cannot drift; the §6.R IPv4 scanner carries a narrow path+range exemption for the `10.0.2.0/24` slirp range scoped to this helper (see `scripts/scan-no-hardcoded-ipv4.sh`).

Usage

```
source scripts/autonomous-qa/lib-emulator.sh
emu_cleanup_orphans
CONTAINER="$(emu_boot default)"
ADB_PORT="$(grep '^ADB_PORT=' scripts/autonomous-qa/.emu-state |
    cut -d= -f2)"
```

```bash
emu_authorize_adb "$CONTAINER"
SERIAL="$(emu_connect "$ADB_PORT")"
emu_wait_boot "$SERIAL" 360
emu_fix_network "$SERIAL"
# ... run tests against $SERIAL ...
emu_teardown
```

Companion files

- `scripts/autonomous-qa/run-matrix.sh`, `run-nav-challenges.sh`, `release-coldstart-canary.sh` — the consumers
- `submodules/containers/pkg/emulator/` — the canonical Go implementation this mirrors
- `docs/scripts/run-emulator-tests.sh.md` — the sibling Containers-driven emulator runner